

# [JA] MID を使用した解約率予測 GLM の改良 - 基本編

## [EN] English Here

Reiji Kozuma<sup>\*</sup>   Ryoichi Asashiba<sup>†</sup>   Yoshihiro Matsumori<sup>‡</sup>   Hirokazu Iwasawa<sup>§</sup>

## 目次

### 1 準備

- 1.1 パッケージの読み込み
- 1.2 データの読み込み
- 1.3 評価関数の準備

### 2 モデルの構築

- 2.1 Reduced Baseline GLM
- 2.2 XGBoost Model
- 2.3 Improved GLM
  - 2.3.1 Functional Decomposition and Feature Importance
  - 2.3.2 Model Refinements
    - I-1: Main Effect of `face_amount`
    - I-2: Main Effect of `risk_class`
    - I-3: Interaction between `premium_mode` and `premium_jump_ratio`
  - 2.3.3 モデル構築

### 3 Statistical Predictive Performance

- 3.1 Reduced Baseline GLM
- 3.2 XGBoost Model
- 3.3 Baseline GLM
- 3.4 Improved GLM

## Acknowledgments

## References

[JA] 概要 [EN] Abstract

---

<sup>\*</sup> Daido Life Insurance Company, [rktkdm@gmail.com](mailto:rktkdm@gmail.com)

<sup>†</sup> Daido Life Insurance Company, [ryoichi.asashiba@gmail.com](mailto:ryoichi.asashiba@gmail.com)

<sup>‡</sup> Sompo Japan Insurance Inc., [ymatsumori1@sompo-japan.co.jp](mailto:ymatsumori1@sompo-japan.co.jp)

<sup>§</sup> Graduate School of Accountancy, Waseda University, [iwahiro@bb.mbn.or.jp](mailto:iwahiro@bb.mbn.or.jp)

[JA] 本ドキュメントは、論文 [において](#)提案した、新たな解釈手法 MID を活用した EV 解約率前提用 GLM の改善過程における基本的な手順を、直ぐに利用可能なコード例の形で示すものである。データセットの前処理、その研究で利用したモデルの構築、およびパッケージ midr を活用した可視化の方法について述べる。

なお、使用するモデルや評価指標、解釈手法の解説については元となる論文に譲ることとし、本ドキュメントではコードを示すことによる具体的な使用方法の説明に注力する。また、EV の簡易計算や、論文のデータの再現のためのドキュメントは、別ファイルで提供する。

コードの多くは元の研究の内容を再現するためのものであり、それらの中には、初学者にとっては理解が難しいコードも存在する。そのため、初学者は、Section 2.3.2 以外の内容については単に記載されているコードを深く考えずにそのまま実行するだけでもさしあたりは問題ない。

[EN] English Here

## 1 準備

### 1.1 パッケージの読み込み

[JA] 本ドキュメントで使用するパッケージを読み込む。

[EN] English Here

```
library(midr)
library(tidyverse)
library(ROCR)
```

### 1.2 データの読み込み

[JA] 最初に、本研究で利用したデータセット Society of Actuaries ([2014](#)) を読み込む<sup>\*1</sup>。

[EN] English Here

```
# install.packages("devtools")
# devtools::install_github("kasaai/insurance")
library(insurance)
library(tidyverse)
df_raw <- lapse_study
```

[JA] 次に、データを利用しやすいように前処理を行う。便宜上追加した ID 列を除けば、元論文で説明したとおりである。

[EN] English Here

```
# ID 列の追加
df_raw <- df_raw %>% mutate(ID = as.integer(rownames(df_raw)))

# lapse_study_year (Convert from string "yyyy-(yyyy+1)" to integer yyyy),
# issue_year, duration
df_raw <- df_raw %>% mutate(
  lapse_study_year = as.integer(substr(lapse_study_year, 1, 4)),
  issue_year =
    if_else(issue_year == "Pre-1991", "-1990", issue_year) %>%
```

---

<sup>\*1</sup> <https://github.com/kevinykuo/insurance/tree/master/data> [JA] からダウンロード可能。[EN] Available for download.

```

    as.ordered(),
    duration = case_when(
      duration == "6-9" ~ "06-09",
      duration == "13+" ~ "13-",
      TRUE ~ duration
    ) %>% as.ordered()
  )

# post_level_premium_structure
df_raw <- df_raw %>% mutate(
  post_level_premium_structure =
    if_else(
      post_level_premium_structure == "1. Premium Jump to ART",
      "ART", "Other"
    ) %>% factor(levels = c("ART", "Other"))

# premium_jump_ratio
df_raw <- df_raw %>% mutate(
  premium_jump_ratio =
    case_when(
      premium_jump_ratio == "Y. Unknown" ~ "Unknown",
      premium_jump_ratio == "X. 24.01 AND UP" ~ "24.01-",
      .default = as.integer(substr(premium_jump_ratio, 4, 5)) %>%
        (function(x) sprintf("%#2d.01-%#2d.00", x, x+1))) %>%
      as.factor()

# Others
levels_risk_class <-
  c("Super-Pref NS", "Pref Best NS", "Pref Resid NS", "Non-Pref NS", "Undiff NS",
    "Pref SM", "Non-Pref SM", "Undiff SM", "Agg/Unknown")
levels_premium_mode <-
  c("Annual", "Semiannual", "Quarterly", "Monthly", "Biweekly", "Unknown/Other")

df_raw <- df_raw %>% mutate(
  premium_mode = premium_mode %>% substring(4) %>%
    factor(levels = levels_premium_mode),
  risk_class = risk_class %>%
    factor(levels = levels_risk_class),
  gender = gender %>%
    factor(levels = c("M", "F"),
          labels = c("male", "female")),
  issue_age = as.ordered(issue_age),
  face_amount = face_amount %>%
    factor(levels = c("A. < 100k", "B. 100k-249k", "C. 250k-999k", "D. 1M +"),
          labels = c("-100K", "100K-250K", "250K-1M", "1M-"),
          ordered = TRUE))

# Drop unused data.
df_raw <- df_raw %>% select(
  ID, lapse_study_year, policy_year,

```

```

    duration,
    issue_year, issue_age, gender, face_amount, risk_class,
    premium_mode, premium_jump_ratio, post_level_premium_structure,
    exposure_count, lapse_count,
    exposure_amount, lapse_amount
  )
df_all <- df_raw %>% filter(exposure_count != 0)
df_all <- df_all %>% select(-lapse_amount) %>%
  rename(exposure = exposure_count) %>%
  rename(lapse = lapse_count)

# 目的変数として、各グループの解約率を計算
df_all <- df_all %>% mutate(lapse_rate = lapse / exposure)

prep_toint_pjratio <- function(df_raw){
  df_raw %>%
    mutate(premium_jump_ratio_unknown =
      if_else(premium_jump_ratio == "Unknown", 1, 0)) %>%
    mutate(premium_jump_ratio =
      fct_recode(premium_jump_ratio,
        "05" = "Unknown", "27" = "24.01-") %>%
      substr(1, 2) %>% as.integer())
}

```

[JA] 後で説明するように、MID による関数分解を行うモデルは 2004-2006 保険年度で学習したもの (Validation Year は 2007) とするので、このデータを抽出して準備する。

[EN] English Here

```

year_midr <- 2006 # Validation Year - 1
df_train_2006 <- df_all %>%
  filter(lapse_study_year >= year_midr-2, lapse_study_year <= year_midr)
df_test_2006 <- df_all %>%
  filter(lapse_study_year == year_midr+1)

```

### 1.3 評価関数の準備

[JA] Log-loss を重み付きで計算する関数の実装を行う。の引数 mean を FALSE とした場合は Log-loss の総和が、TRUE とした場合は加重平均値が出力される。

[EN] English Here

```

xlogy <- function(x, y){
  if_else(x == 0, 0, x * log(y))
}
calc_logloss <- function(y_pred, y_act, w = NULL, mean = T){
  losses <- - xlogy(y_act, y_pred) - xlogy(1 - y_act, 1 - y_pred)
  if(is.null(w)){
    if(mean){
      return(mean(losses))
    }else{

```

```

        return(sum(losses))
    }
}
if(mean){
    return(sum(w * losses) / sum(w))
}else{
    return(sum(w * losses))
}
}

```

[JA] 続いて、ROC-AUC を重み付きで計算する関数の実装を行う。参考: [ROCR/R/prediction.R at master · ipa-tys/ROCR](#)

[EN] [English Here](#)

```

calc_auc <- function(y_pred, y_act, w = NULL, verbose = FALSE){
  predictions <- y_pred
  pred.order <- order(predictions, decreasing=TRUE)
  predictions.sorted <- predictions[pred.order]
  tp <- cumsum(y_act[pred.order] * w[pred.order])
  fp <- cumsum((1-y_act[pred.order]) * w[pred.order])
  dups <- rev(duplicated(rev(predictions.sorted)))

  tp <- c(0, tp[!dups])
  fp <- c(0, fp[!dups])
  cutoffs <- c(Inf, predictions.sorted[!dups])

  n.pos <- sum(y_act*w)
  n.neg <- sum((1-y_act) * w)
  fn <- n.pos - tp
  tn <- n.neg - fp
  n.pos.pred <-
  n.neg.pred <-

  pred <- new("prediction", predictions=list(predictions),
            labels=list(c("0", "1")),
            cutoffs=list( cutoffs ),
            fp=list( fp ),
            tp=list( tp ),
            fn=list( fn ),
            tn=list( tn ),
            n.pos=list( n.pos),
            n.neg=list( n.neg),
            n.pos.pred=list(tp + fp),
            n.neg.pred=list(tn + fn))
  auc <- performance(pred, "auc")
  if (verbose){
    perf <- performance(pred, "tpr", "fpr")
    list(auc = auc, curve = perf)
  }else{
    as.numeric(auc@y.values)
  }
}

```

```
}  
}
```

## 2 モデルの構築

### 2.1 Reduced Baseline GLM

[JA] 本研究では複数のモデルを取り扱う。モデルの前処理もモデルによって異なることから、前処理部分をモデル固有の関数として実装することで、コードの見通しをよくする。

[EN] English Here

[JA] まず、`issue_age` に対しては、SOA のレポート P. 39 に記載された代表年齢に変換する関数を作成する。

[EN] English Here

```
prep_toint_age <- function(df_raw){  
  df_raw %>%  
    mutate(issue_age = case_match(issue_age,  
      "0-19" ~ 9.5,  
      "20-29" ~ 26.3,  
      "30-39" ~ 34.7,  
      "40-49" ~ 44.2,  
      "50-59" ~ 53.7,  
      "60-69" ~ 63.3,  
      "70+" ~ 72.3  
    ))  
}
```

[JA] 次に、SOA の作成した GLM と同様の前処理を行う関数を作成する。ただし、ここで構築するのは `face_amount` を取り除いた the reduced Baseline GLM である。

[EN] English Here

```
prepfun_soa_xy <- function(df_tmp){  
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%  
    mutate(  
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),  
      risk_class_group = risk_class %>%  
        factor(levels = levels_risk_class,  
              labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),  
      premium_mode_group = premium_mode %>%  
        factor(levels = levels_premium_mode,  
              labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",  
                        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),  
      premium_jump_ratio_group = cut(  
        if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),  
        breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%  
    select(duration, post_level_premium_structure,  
           issue_age, issue_age_sq, issue_age_log, risk_class_group,  
           premium_mode_group, premium_jump_ratio_group, exposure, lapse)  
}
```

[JA] 以上の前処理を実際に適用したデータフレームを作成しておく.

[EN] English Here

```
df_train_2006_soa <- df_train_2006 %>% prepfun_soa_xy
df_test_2006_soa <- df_test_2006 %>% prepfun_soa_xy
```

[JA] 最後に、学習データに対して `glm()` 関数を適用し、モデルを構築する. `duration : post_level_premium_structure` の記述は、これらについて層化したモデルとしていることを表す.

[EN] Excluding preprocessing, the R code for model construction is shown below. The syntax `duration : post_level_premium_structure` represents stratification across these factors.

[JA] `duration : post_level_premium_structure` の指定は計画行列が巨大になることから、層ごとに別々の GLM を構築するよりも計算時間がかかる. この観点で元論文で掲載した精度評価には、層ごとに別々の GLM を構築したものを使用している. 両者は本質的に同等なモデルであるが、1 件も解約が発生しないケースの予測においてわずかに出力される確率値が異なっている. 従って、本ドキュメントによる実装は元論文の結果を厳密には再現しない

[EN] Specifying `duration : post_level_premium_structure` creates a large design matrix. In practice, fitting separate GLMs for each stratum is computationally faster. For the evaluation results in xxx and xxx, separately fitted GLMs are used. While mathematically equivalent, slight numerical variations in predicted probabilities occur in strata with zero observed lapses.

```
model_soa_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  ↪ post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_soa)
```

[JA] 学習データで現れなかったグループがテストデータには現れることがあるという点に注意が必要である. そのようなデータに対しても予測できるよう、テストデータにも前処理が必要となる. 今回使用する年度のデータに対しては、次のような前処理が必要となるので、それを関数として作成する.

[EN] English Here

```
prepfun_soa_xy_test_2006 <- function(df_tmp){
  df_tmp %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "10" &
        premium_jump_ratio_group == "[20,99]", "[16,20]",
      premium_jump_ratio_group)
  ) %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "12" &
        premium_jump_ratio_group == "[16,20]", "[12,16]",
      premium_jump_ratio_group)
  ) %>% mutate(
```

```

    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "13-" &
        premium_jump_ratio_group == "[12,16)", "[10,12)",
      premium_jump_ratio_group)
  ) %>% mutate(
    risk_class_group = if_else(
      post_level_premium_structure == "Other" & duration == "11" &
        risk_class_group == "Super-Pref NS", "NS",
      risk_class_group)
  )
}

```

## 2.2 XGBoost Model

[JA] 次に、GLM の改良に利用するブラックボックスモデルとして XGBoost による予測モデルを構築する。

[EN] Next, we train an XGBoost predictive model as the high-performing black-box benchmark for refining the GLM.

[JA] `xgb.cv()` 関数などのパラメータ `params` に与えるパラメータについては次のとおりである。パラメータ `objective` に `"reg:logistic"` を指定し、ロジスティック損失関数を `objective` として確率を出力させる。また、過学習防止のために木の複雑さを設定する必要がある。その交差検証での判定基準として、パラメータ `eval_metric` を `"logloss"` に指定する。加えて、ハイパーパラメータを設定する。本研究では、`tune` パッケージを利用して事前にチューニングした<sup>\*2</sup>。

[EN] The parameter settings passed to `xgb.cv()` and related functions are configured as follows. Setting `objective = "reg:logistic"` specifies a logistic objective with probability output. To control overfitting, `eval_metric = "logloss"` is used for cross-validation stopping criteria. Hyperparameters (such as maximum depth and learning rate) are tuned in advance using the `tune` package<sup>\*3</sup>.

```

library(xgboost)
get_mm <- function(df){
  df <- df %>%
    select(setdiff(names(df),c(
      "ID", "lapse_study_year", "policy_year", "issue_year",
      "exposure_amount", "lapse", "exposure", "lapse_rate"))) %>%
    prep_toint_pjratio
}

```

<sup>\*2</sup> [JA] learning rate は 0.02 に固定した。その他のハイパーパラメータは、2005 年から 2007 年の学習データを用いた 4 分割交差検証によって一度だけ調整され、ローリングウィンドウ全体を通じて固定した。一方、ブースティングの反復回数は、各ウィンドウについて、そのウィンドウに対応する学習データ内での 4 分割交差検証により個別に決定した。[EN] The learning rate was fixed at 0.02. The other hyperparameters were tuned once using four-fold cross-validation on the 2005–2007 training data and then held fixed across rolling windows, while the number of boosting rounds was selected separately for each window by four-fold cross-validation within the corresponding training data.

<sup>\*3</sup> [JA] learning rate は 0.02 に固定した。その他のハイパーパラメータは、2005 年から 2007 年の学習データを用いた 4 分割交差検証によって一度だけ調整され、ローリングウィンドウ全体を通じて固定した。一方、ブースティングの反復回数は、各ウィンドウについて、そのウィンドウに対応する学習データ内での 4 分割交差検証により個別に決定した。[EN] The learning rate was fixed at 0.02. The other hyperparameters were tuned once using four-fold cross-validation on the 2005–2007 training data and then held fixed across rolling windows, while the number of boosting rounds was selected separately for each window by four-fold cross-validation within the corresponding training data.

```

    model.matrix(~ ., df)
  }
df_train_2006_mm <- df_train_2006 %>% get_mm
constant_cols_2006 <- which(apply(df_train_2006_mm, 2, function(col) all(col == col[1])))
dm_train_2006 <- xgb.DMatrix(
  data = df_train_2006_mm[, -constant_cols_2006, drop = FALSE],
  label = df_train_2006 %>% pull(lapse_rate),
  weight = df_train_2006 %>% pull(exposure))

```

```

params_xgb <- list(
  objective = "reg:logistic",
  eval_metric = "logloss", #Used in CV
  # Hyper parameters. Determined by Bayesian optimization using the "tune" package.
  gamma = 2.966439e-08, #loss_reduction
  eta = 0.02, #learn_rate
  max_depth = 10, #tree_depth
  min_child_weight = 10, #min_n
  subsample = 0.9976594, #sample_size
  colsample_bynode = 6/31 #mtry / count of predictors
)

```

[JA] 以上の準備のもと、`xgb.cv()` 関数により交差検証で木の複雑さ `nrounds` を決定し、`xgb.train()` 関数で予測モデルの構築を行う。

[EN] With these preparations, the optimal number of boosting rounds (`nrounds`) is determined via cross-validation using `xgb.cv()`, and the final prediction model is trained using `xgb.train()`.

```

set.seed(2025)
# Determine the optimal number of boosting rounds by CV
cv_xgboost_2006 <- xgb.cv(data = dm_train_2006, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_2006$early_stop$best_iteration

# Fit model
set.seed(2025)
model_xgboost_2006 <- xgb.train(data = dm_train_2006, nrounds = nrounds,
  params = params_xgb)

```

[JA] この XGBoost が GLM よりも精度の高いモデルであることは別途検証が必要であるが、ここでは記載を省略する。

[EN]

## 2.3 Improved GLM

[JA] ここまでで構築した XGBoost の予測結果を MID を用いて観察し、観察結果を参考にして GLM を改良する。以下では、Section 2.1 で構築した、意図的に `face_amount` を除外したモデル (**reduced baseline GLM**) を改良の出発点とする。

[EN] English Here

[JA] つづく節では、改良の手順の一部をコード例を示しながら説明する。

[EN] English Here

### 2.3.1 Functional Decomposition and Feature Importance

[JA] 関数分解を行うモデルの対象となるデータを用意する。ここで、本研究では検証年度に応じて複数のモデルを構築しているが、MID を観察するモデルは 2004-2006 保険年度で学習したモデルとし、関数分解に用いるインスタンスも、その学習データとする。これは、SOA の GLM と XGBoost の間の予測の差が一定程度大きく、かつ解約の予測が外れやすい時期にはまだ差し掛かっていないことによる<sup>\*4</sup>。

[EN] We prepare the dataset for performing functional decomposition. While multiple models are trained across different time windows in this study, MID inspection is conducted on the model trained on the 2004–2006 policy years. This window is selected because the prediction gap between GLM and XGBoost is sufficiently pronounced without yet being confounded by extreme market shifts<sup>\*5</sup>.

[JA] 重みを指定するためのデータや、予測に使用する XGBoost モデル固有の関数 (`pred.fun`) を用意する必要があるため、それをここで定義する。

[EN] English Here

```
df_train_2006_xw <- df_train_2006 %>%
  select(-ID, -lapse_study_year, -policy_year, -issue_year, -exposure_amount, -lapse,
    ↪ -lapse_rate)
df_train_2006_w <- df_train_2006_xw %>% pull(exposure)
predfun_xgb <- function(model, newdata){
  df_mm <- newdata %>% get_mm
  df_mm <- df_mm[, -constant_cols_2006, drop = FALSE]
  predict(model, df_mm)
}
```

[JA] まず最初のステップとして、XGBoost の予測値を関数分解するため、`midr::interpret()` 関数を実行する<sup>\*6</sup>。MID of order 2 までの分解が実行可能だが、今回のケースでは分解のときに、すべての交互作用項を用意すると singular fit の回避が困難であるため、一部の交互作用項を省略した形の分解を行う。

[EN] As the first step, we execute `midr::interpret()` to perform functional decomposition on XGBoost predictions<sup>\*7</sup>. While MID of order 2 computes up to two-way interactions, we omit a subset of interactions to prevent singular matrix fits in the decomposition step.

<sup>\*4</sup> [JA] 元論文に記載する EV 変動要因分析による前提実績差異を確認すると、2008 保険年度以降は XGBoost でも差異が大きくなっており、将来に向かっての解約の予測が困難であることを示唆している。[EN] As shown in xxx, experience variances increase substantially in magnitude across all models from the 2008 policy year onward, reflecting heightened forecasting difficulty during the financial crisis.

<sup>\*5</sup> [JA] 元論文に記載する EV 変動要因分析による前提実績差異を確認すると、2008 保険年度以降は XGBoost でも差異が大きくなっており、将来に向かっての解約の予測が困難であることを示唆している。[EN] As shown in xxx, experience variances increase substantially in magnitude across all models from the 2008 policy year onward, reflecting heightened forecasting difficulty during the financial crisis.

<sup>\*6</sup> [JA] `midr` パッケージの使用の詳細は <https://cran.r-project.org/web/packages/midr/readme/README.html> を参照。[EN] For implementation details of the `midr` package, see <https://cran.r-project.org/web/packages/midr/readme/README.html>.

<sup>\*7</sup> [JA] `midr` パッケージの使用の詳細は <https://cran.r-project.org/web/packages/midr/readme/README.html> を参照。[EN] For implementation details of the `midr` package, see <https://cran.r-project.org/web/packages/midr/readme/README.html>.

```
mid_xgb <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+
        face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_xgboost_2006, pred.fun = predfun_xgb, link = "logit",
  # Penalty factor for pseudo-smoothing; helps avoid singular fits.
  lambda = 0.01,
  verbosity = 0 # Stop when a singular fit is encountered.
)
```

[JA] 可視化のステップに入る前に、見た目を整えるために、幾つかの準備を行う。これは分析上必須ではない。

[EN]

```
# Display border lines.
theme_bw16 <- theme_bw(base_size = 16)
# Display the text on the X-axis vertically.
theme_axisxvert <- theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
theme_set(theme_bw16 + theme_axisxvert)
# Set default color theme (ggmid's argument "theme").
options(midr.sequential = "Oslo")
# Modify appearance of scatter plots.
update_geom_defaults("point", list(alpha = 0.5, size = 2.0, shape = 16))
# Utility function to add labels
add_label <- function(data){
  geom_label(
    data = data, aes(x = x, y = y, label = label),
    size = 5, fill = "white", colour = "black",
    label.size = 0.1, label.padding = unit(0.1, "lines"), fontface = "bold"
  )
}
```

[JA] 次に特徴量とそれらの間の交互作用の重要度を可視化する。結果は Figure 1 のとおり。

[EN] Next, we visualize the importance of feature main effects and two-way interactions. The feature effect importance heatmap is shown in Figure 1.

```
mid_xgb %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap")
```

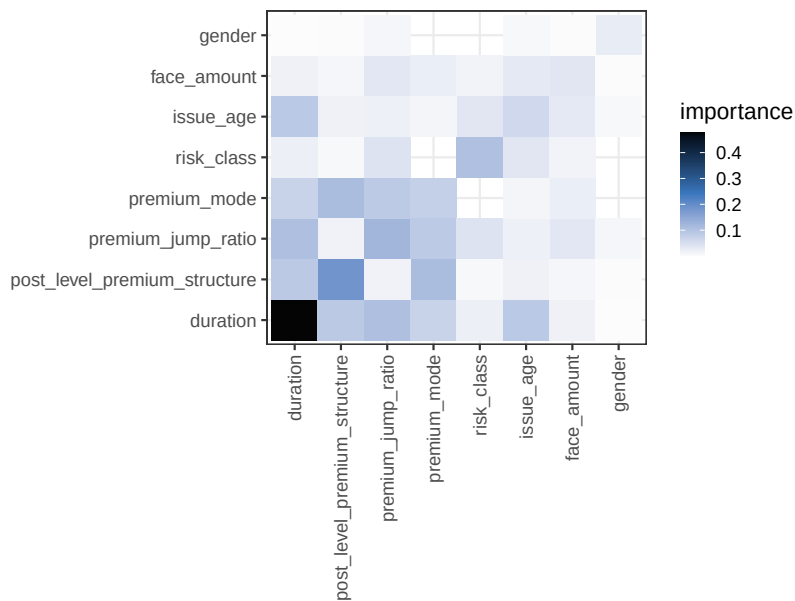


Figure1: Feature Effect Importance

[JA] 関数分解を行う対象をベースラインの GLM と XGBoost の予測値の差に変更することで、両者の予測の差異がどの説明変数に關係して現れるのかを可視化することもできる。結果は Figure 2 のとおり。

[EN] By changing the decomposition target to the prediction difference between the reduced baseline GLM and XGBoost, we can directly visualize which features drive the discrepancies between the two models. The resulting importance plot for the difference model is shown in Figure 2.

[JA] いずれも、分解の対象となる予測値が確率の値そのものではなく線形予測子のもの（logit 変換を介した後の数値）であることに注意する。

[EN] Note that the decomposition is applied to the linear predictor scale (logit-transformed values) rather than raw probability predictions.

```
fn_logit <- function(p){log(p/(1-p))} # link function
fn_expit <- function(x){1/(1+exp(-x))} # inverse of the link function

# pred.fun for the SOA GLM model
predfun_soa_2006 <- function(model, newdata){
  predict(model,
    newdata %>% mutate(lapse = 0) %>% #prevent error in prepfun_soa_xy
    prepfun_soa_xy %>% prepfun_soa_xy_test_2006,
    type = c("response"))
}

# pred.fun for "difference" models
predfun_diff <- function(model, newdata){
  p_target <- model$predfun_glm(model$model_glm, newdata)
  p_base <- model$predfun_xgb(model$model_xgb, newdata)
  fn_expit(fn_logit(p_target) - fn_logit(p_base)) #apply the link function
}
```

```

# Create the "difference" model
model_diff <- list()
model_diff$model_xgb <- model_xgboost_2006
model_diff$model_glm <- model_soa_2006
model_diff$predfun_xgb <- predfun_xgb
model_diff$predfun_glm <- predfun_soa_2006

# Execute midr::interpret for the "difference" model
mid_diff <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_diff, pred.fun = predfun_diff,
  link = "logit", lambda = 0.01,
  verbosity = 0
)

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("post_level_premium_structure",
        "risk_class", "premium_mode", "premium_mode", "premium_mode",
        "issue_age", "risk_class", "premium_jump_ratio", "issue_age",
        ↵ "premium_jump_ratio", "gender"),
  y = c("post_level_premium_structure",
        "risk_class", "premium_mode", "premium_jump_ratio",
        ↵ "post_level_premium_structure",
        "issue_age", "issue_age", "risk_class", "face_amount", "face_amount", "gender"),
  label = c("X", "A-2", "A-3", "A-3", "A-4", "A-5", "A-6", "A-7", "A-8", "A-9", "A-10")
))

```

```

#display feature effect imporatanace plot
mid_diff %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap") + add_label_tmp

```

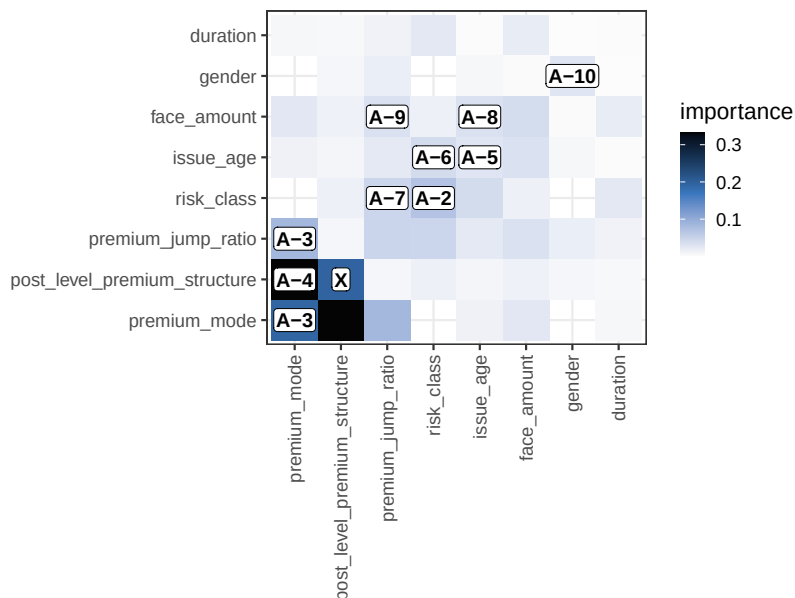


Figure2: Feature Effect Importance of the difference model

[JA] 今回の GLM は層化されているため、ある層に特化した分析を行うことも考えられる。たとえば、レポートで GLM の構築対象とした `duration == 10`, `post_level_premium_structure == "ART"` にデータを限定して関数分解を行うと Figure 3 のとおり。

[EN] Because the GLM is stratified, focused analysis on specific strata can also be conducted. For example, restricting the decomposition to the `duration == 10` and `post_level_premium_structure == "ART"` stratum yields Figure 3.

```
# Create filtered data.
df_train_2006_xw_d10ART <- df_train_2006_xw %>%
  filter(duration == 10, post_level_premium_structure == "ART")
df_train_2006_w_d10ART <- df_train_2006_xw_d10ART %>% pull(exposure)

# midr::interpret.
mid_xgb_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
  model = model_xgboost_2006, pred.fun = predfun_xgb,
  link = "logit", lambda = 0.01, verbosity = 0
)
mid_diff_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
  model = model_diff, pred.fun = predfun_diff,
  link = "logit", lambda = 0.01, verbosity = 0
)
```

```

)

# Display feature effect importance plot.
mid_xgb_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("XGBoost")

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("premium_jump_ratio", "face_amount", "risk_class", "premium_mode", "risk_class"),
  y = c("premium_jump_ratio", "face_amount", "risk_class", "premium_jump_ratio",
        ↪ "premium_jump_ratio"),
  label = c("Y", "B-1", "B-2", "B-3", "B-7")
))

# Display feature effect importance plot.
mid_diff_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("GLM - XGBoost") + add_label_tmp

```

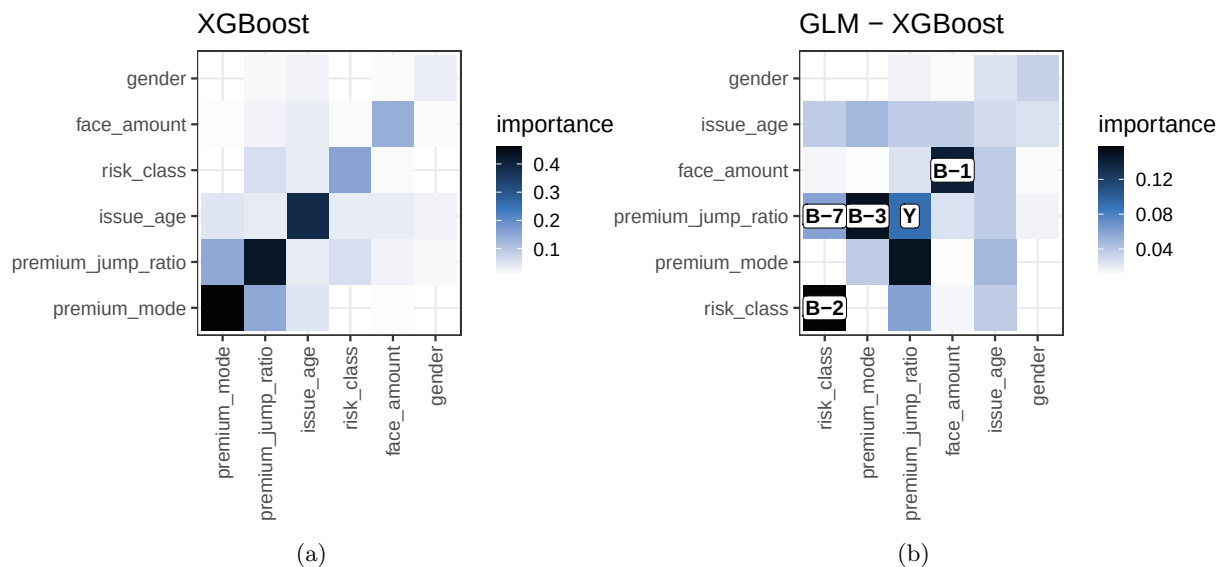


Figure3: Feature Effect Importance in Duration = 10, ART

### 2.3.2 Model Refinements

[JA] GLM と XGBoost の予測結果の差を関数分解して得られる特徴量重要度の図, すなわち Figure 2 と Figure 3b を確認し, 差異が大きいと考えられる説明変数に着目して分析を行う.

[EN] We examine the difference importance plots (Figure 2 and Figure 3b) and focus our analysis on features showing large prediction discrepancies.

[JA] なお, 差異が大きい説明変数全てで, GLM のモデルの改良点が見つかるとは限らない. たとえば,

Figure 2 の X によると、`post_level_premium_structure` の主効果で差異が大きいという表示があるが、GLM はこの説明変数で層化されており、この主効果の差をモデルに明示的に反映することは出来ない。またたとえば、Figure 3b の Y によると、`premium_jump_ratio` の主効果で差異が大きいという表示がある。reduced baseline GLM ではこの説明変数についてベニングの上でダミー変数化している。そのベニング方法の調節が考えられる程度で、関数分解の形からモデル改善に有用な直接的な示唆を得るのは難しいと考えられる。

[EN] Not all discrepancies lead directly to actionable GLM modifications. For instance, point X in Figure 2 indicates a discrepancy in `post_level_premium_structure`, but since the GLM is already stratified by this factor, main effect modifications cannot be explicitly added. Similarly, point Y in Figure 3b indicates a discrepancy in `premium_jump_ratio`, which is already binned into dummy variables in the reduced baseline GLM.

[JA] 以上を除き、おおむね差異が大きいように思われる点から順番に吟味していく。

[EN] Excluding these cases, we evaluate the high-discrepancy components in order of magnitude.

[JA] 本文では、冒頭の 3 項目について改良手順を述べる。他の詳細は本ドキュメントでは割愛する。

[EN] English Here

#### I-1: Main Effect of `face_amount`

[JA] Figure 3b の B-1 によると、`face_amount` の主効果で差異が大きいという表示がある。この説明変数は reduced baseline GLM では採用されていないが、この図からは解約率との何らかの関係性が示唆される。そこで、XGBoost の予測値の関数分解で得られた関数を確認する。

[EN] Point B-1 in Figure 3b highlights a prominent discrepancy in the main effect of `face_amount`. While omitted in our reduced baseline GLM, MID indicates its strong influence on lapse predictions.

[JA] 関数分解で得られた関数をプロットするには、`midr::ggmid()` 関数を使用する。

[EN] English here.

[JA] `duration == 10`, ART における、`face_amount` の主効果の関数項をプロットすると Figure 4 のとおりで、高額契約ほど解約率が高いことが示唆される。

[EN] Plotting the main effect function term of `face_amount` for `duration == 10`, ART (Figure 4) reveals that higher face amounts correspond to higher predicted lapse rates.

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw, term = "face_amount")
```

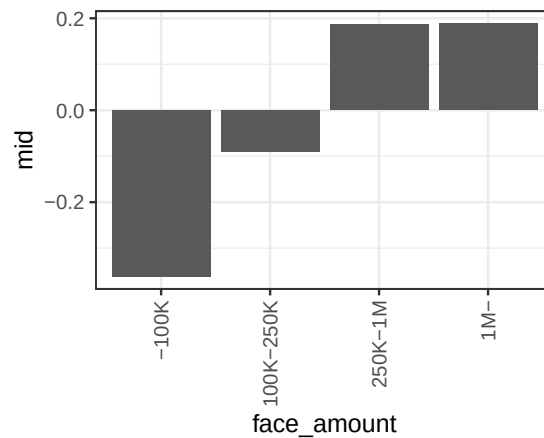


Figure4: Main Effect Plot of face\_amount

[JA] Figure 3b および Figure 4 は duration == 10, ART に限った分析を示している. そこで, 他の duration での様子も同時に可視化するため, face\_amount と duration の 2 変数を組み合わせた効果を可視化すると Figure 5 のようになる.

[EN] To examine whether this relationship holds across other durations, Figure 5 displays the compound effect (main effects + interaction) of face\_amount and duration.

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
  term = "face_amount:duration", main.effects = TRUE,
  type = "compound") #add sample points to the plot
```

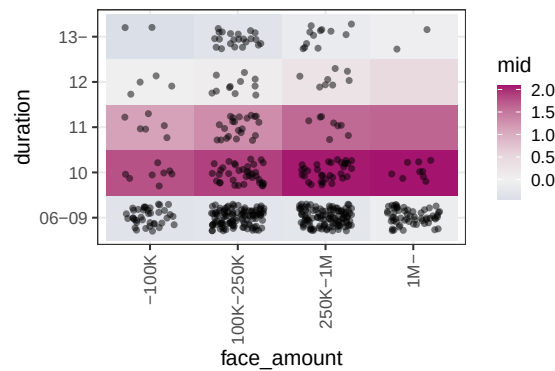


Figure5: Main Effect + Interaction Plot of duration and face\_amount

[JA] Figure 5 により, duration <= 9 では高額契約ほど解約率が低く, duration >= 10 ではその逆であることがわかる. GLM に説明変数として face\_amount を追加することで, 精度改善に貢献するものと考えられる.

[EN] English here.

## I-2: Main Effect of risk\_class

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                        term = "risk_class:duration", main.effects = TRUE,
                        type = "compound")
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                        term = "premium_mode:duration", main.effects = TRUE,
                        type = "compound")
```

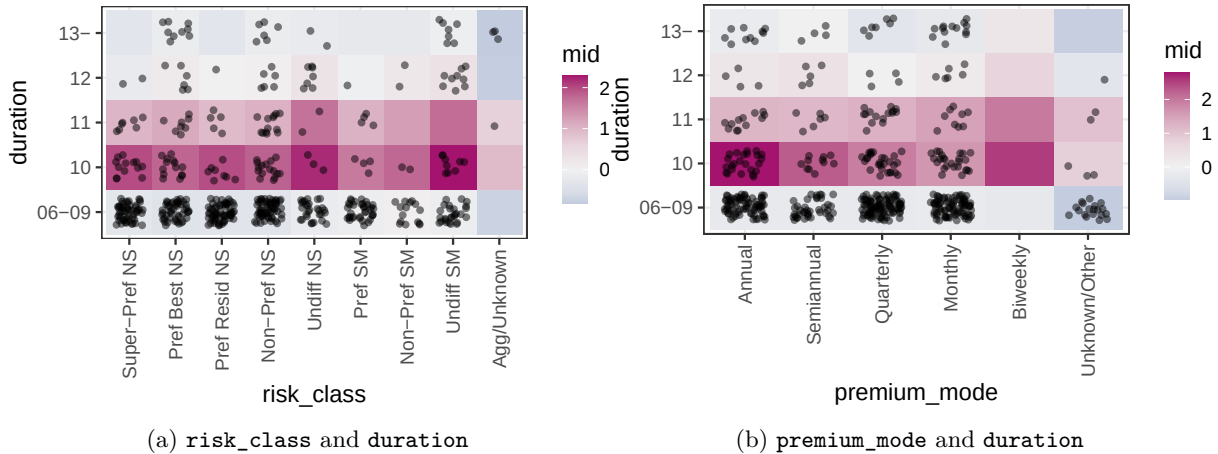


Figure6: Main Effects + Interaction Plots

[JA] Figure 2 の A-2 および Figure 3b の B-2 によると, `risk_class` の主効果で差異が大きいという表示がある. そこで I-1 と同じように `risk_class` と `duration` の 2 変数を組み合わせた効果を可視化すると Figure 6a のとおり. GLM は "Super-Pref NS", それ以外の "NS", その他の 3 区分でのみ区分けしていたが, "Undiff NS", "Undiff SM" はそれ以外の "NS", "SM" とは異なる傾向を示している. さらに, "Agg/Unknown" はどれも異なる傾向を示している. 加入時の健康状態という意味では, "Undiff" が付されているものはそうでないものは一定程度類似しているはずであるが, 実際は解約率に影響が現れる形で契約者の性質が異なる (例えば保険会社, 地域, チャンネル等の特性の差異がある) と考えられる. "Agg/Unknown" も同様に, ある特殊な契約群団を特定できる要素となっている可能性がある. 健康状態そのものが解約動向にどのような影響を及ぼすかは判断の難しいところではあるが, この図を観察する限りでは解約率には差があるため, これを区分することは解約率予測精度向上に資するものと考えられる. そこで, GLM でも "Undiff NS", "Undiff SM", "Agg/Unknown" は独立した level として設定する変更を採用する<sup>\*8</sup>.

[EN] Points A-2 (Figure 2) and B-2 (Figure 3b) highlight a large discrepancy in `risk_class`. Plotting the compound effect of `risk_class` and `duration` (Figure 6a) reveals that while the baseline GLM

<sup>\*8</sup> [JA] 実際は, 層化によって各層の学習データが小さくなっており, 対象の level に属するデータが非常に少ないと過学習に陥る. 予測精度を向上させるため, 一定程度の規模がある場合のみ区分することとした. 区分の判断基準は次のとおりで, 閾値は予測精度が向上するかによって決定した. (i) If "Super-Pref NS" is smaller than 50, then merge "Super-Pref NS" and "NS". (ii) If "Undiff NS" or NS is smaller than 700, then merge "Undiff NS" and "NS". (iii) If "Agg/Unknown" is smaller than 100, then merge "Undiff SM" and "Agg/Unknown". (iv) If "SM" is smaller than 700, then merge "Undiff SM" and "SM". [EN] To avoid overfitting in small strata, collapsing rules are applied based on sample counts: (i) Merge "Super-Pref NS" with "NS" if count < 50; (ii) Merge "Undiff NS" and "NS" if either count < 700; (iii) Merge "Agg/Unknown" with "Undiff SM" if count < 100; (iv) Merge "SM" with "Undiff SM" if count < 700.

consolidated risk classes into three broad categories, undifferentiated groups ("Undiff NS", "Undiff SM") and "Agg/Unknown" exhibit distinct lapse trends. Therefore, we update the GLM to treat "Undiff NS", "Undiff SM", and "Agg/Unknown" as independent factor levels <sup>\*9</sup>.

### I-3: Interaction between premium\_mode and premium\_jump\_ratio

[JA] Figure 2 の A-3 および Figure 3b の B-3 によると, premium\_mode と premium\_jump\_ratio の交互作用で差異が大きいという表示がある. この交互作用の確認の前に, premium\_mode の主効果について検討するため, risk\_class と duration の2変数を組み合わせた効果を可視化すると Figure 6b のとおり. GLM は "Semiannual", "Quarterly" を統合してひとつの level としていたが, 実際は解約率に差が認められるため, これらを分離する変更を採用する. 一方, "Biweekly" は "Monthly" と同じ区分となっているが, 群団として非常に小さいため区分の分離は行わない.

[EN] Points A-3 and B-3 indicate a significant discrepancy in the interaction between premium\_mode and premium\_jump\_ratio. Examining the main effect of premium\_mode across durations (Figure 6b) shows clear differences between "Semiannual" and "Quarterly" modes, prompting us to separate them into distinct levels in the GLM.

[JA] 次に, premium\_mode と premium\_jump\_ratio の交互作用の確認のため, duration == 10, ART にてこの2変数の効果をプロットすると Figure 7 のとおり. Figure 7a は主効果の関数値を加算している一方, Figure 7b は交互作用の関数値のみをプロットしている.

[EN] Next, to examine the interaction between premium\_mode and premium\_jump\_ratio, Figure 7 plots their combined effects at duration == 10, ART. Specifically, Figure 7a displays both the main effects and the interaction term, whereas Figure 7b isolates the interaction effect alone <sup>\*10</sup>.

[JA] グラフに主効果を加算するかどうかは, 引数 main.effects によって制御できる.

[EN] English Here.

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode", main.effects = TRUE,
                               type = "compound") +
  ggtitle("Duration = 10, ART, Main Effects + Interaction")
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode",
                               type = "compound") +
```

<sup>\*9</sup> [JA] 実際は, 層化によって各層の学習データが小さくなっており, 対象の level に属するデータが非常に少ないと過学習に陥る. 予測精度を向上させるため, 一定程度の規模がある場合のみ区分することとした. 区分の判断基準は次のとおりで, 閾値は予測精度が向上するかによって決定した. (i) If "Super-Pref NS" is smaller than 50, then merge "Super-Pref NS" and "NS". (ii) If "Undiff NS" or NS is smaller than 700, then merge "Undiff NS" and "NS". (iii) If "Agg/Unknown" is smaller than 100, then merge "Undiff SM" and "Agg/Unknown". (iv) If "SM" is smaller than 700, then merge "Undiff SM" and "SM". [EN] To avoid overfitting in small strata, collapsing rules are applied based on sample counts: (i) Merge "Super-Pref NS" with "NS" if count < 50; (ii) Merge "Undiff NS" and "NS" if either count < 700; (iii) Merge "Agg/Unknown" with "Undiff SM" if count < 100; (iv) Merge "SM" with "Undiff SM" if count < 700.

<sup>\*10</sup> [JA] 関数項が過度に複雑にならないよう, この交互作用項では premium\_jump\_ratio のビンングを breaks = c(0,1,2,5,99) とする. また, 学習データに実在データが無いビンでの予測が必要な場合, 学習データがある最も近い premium\_jump\_ratio のビンに振り分ける. その際, より高いビンがあればそちらを, そうでなければより低いビンを採用する. [EN] To prevent excessive model complexity, coarse binning (breaks = c(0,1,2,5,99)) is applied to premium\_jump\_ratio for this specific interaction term. If a prediction is required for a bin that lacks observed data in the training set, it is remapped to the nearest bin containing observed data, prioritizing a higher bin if available, or a lower bin otherwise.

```
ggtitle("Duration = 10, ART, Interaction Only")
```

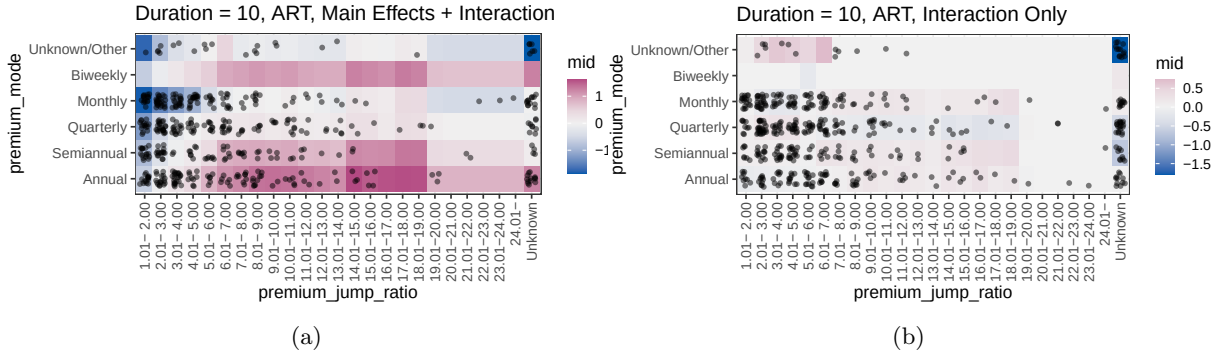


Figure7: Effects of premium\_jump\_ratio and premium\_mode

[JA] Figure 7b によると, "Quarterly" の区分では, premium\_jump\_ratio が高くなるほど解約率が高くなる, といった関係性が弱くなることが示唆されている. そこで, duration == 10 かつ "Quarterly" の場合にのみ, premium\_mode と premium\_jump\_ratio の交互作用項を追加する変更を採用する<sup>\*11</sup>.

[EN] As indicated in Figure 7b, for the "Quarterly" mode, the relationship where higher premium\_jump\_ratio leads to higher lapse rates is substantially weakened. Therefore, we adopt a modification adding an interaction term between premium\_mode and premium\_jump\_ratio exclusively when duration == 10 and premium\_mode == "Quarterly".

### 2.3.3 モデル構築

[JA] 修正 I-1 により face\_amount を復元した「baseline GLM」, つまり SOA が構築したものと同等のモデルを次のとおり構築する. 前処理のための関数をごく一部修正するだけでよい.

[EN] English Here.

```
prepfun_modi1_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
  mutate(
    issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
    risk_class_group = risk_class %>%
      factor(levels = levels_risk_class,
            labels = c("Super-Pref NS","NS","NS","NS","NS","SM","SM","SM","SM")),
    premium_mode_group = premium_mode %>%
      factor(levels = levels_premium_mode,
            labels = c("Annual","Semiannual/Quarterly","Semiannual/Quarterly",
                      "Monthly/Biweekly","Monthly/Biweekly","Unknown/Other")),
```

<sup>\*11</sup> [JA] 関数項が過度に複雑にならないよう, この交互作用項では premium\_jump\_ratio のビンングを breaks = c(0,1,2,5,99) とする. また, 学習データに実在データが無いビンでの予測が必要な場合, 学習データがある最も近い premium\_jump\_ratio のビンに振り分ける. その際, より高いビンがあればそちらを, そうでなければより低いビンを採用する. [EN] To prevent excessive model complexity, coarse binning (breaks = c(0,1,2,5,99)) is applied to premium\_jump\_ratio for this specific interaction term. If a prediction is required for a bin that lacks observed data in the training set, it is remapped to the nearest bin containing observed data, prioritizing a higher bin if available, or a lower bin otherwise.

```

    premium_jump_ratio_group = cut(
      if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
      breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%
select(duration, post_level_premium_structure,
       issue_age, issue_age_sq, issue_age_log, risk_class_group,
       premium_mode_group, premium_jump_ratio_group,
       face_amount, #I-1
       exposure, lapse)
}

```

```

df_train_2006_modi1 <- df_train_2006 %>% prepfun_modi1_xy
df_test_2006_modi1 <- df_test_2006 %>% prepfun_modi1_xy

model_modi1_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  ↪ post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_modi1)

```

[JA] さらに、元論文における 10 個の改良を全て施したモデルの構築方法は次のとおり。改良の大部分は、前処理のための関数の中に実装される。

[EN] English Here.

```

prepfun_mod_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
  mutate(
    premium_mode_oo = #I-4
    if_else(
      duration >= "10" & post_level_premium_structure == "Other" &
      premium_mode == "Unknown/Other", 1, 0),
    post_level_premium_structure = if_else( #I-4
      premium_mode_oo == 1, "ART", post_level_premium_structure),
    premium_mode = if_else( #I-4
      premium_mode_oo == 1, "Monthly", premium_mode),
    issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
    issue_age_70 = if_else(issue_age >= 70 & duration == "06-09", 1, 0), #I-5
    issue_age_d10ART = if_else( #I-8
      (duration == "06-09" | duration == "10") & post_level_premium_structure == "ART",
      issue_age, 0
    ),
    # risk_class_group = risk_class %>% #cf) non #I-2
    # factor(levels = levels_risk_class,
    #       labels = c("Super-Pref NS","NS","NS","NS","NS","SM","SM","SM","SM")),
    risk_class_group = risk_class %>% #I-2
    factor(levels = levels_risk_class,
          labels = c("Super-Pref NS","NS","NS","NS","Undiff NS",
                    "SM","SM","Undiff SM","Agg/Unknown")),
    risk_class_group = if_else( #I-2 (adjusted to 2006-train)

```

```

risk_class_group == "Undiff NS" &
  (duration != "06-09" | post_level_premium_structure != "ART"),
"NS",
if_else(
  risk_class_group == "Undiff SM" &
    (duration >= "11" & post_level_premium_structure == "ART" |
     duration <= "12" & post_level_premium_structure == "Other") |
    risk_class_group == "Agg/Unknown" &
    (duration <= "11" & post_level_premium_structure == "ART"),
  "SM",
  risk_class_group
)
),
risk_class_int = factor( #I-6 (adjusted to 2006-train)
  if_else(
    (risk_class == "Undiff SM" | risk_class == "Non-Pref NS") & duration == "06-09" |
    (risk_class == "Non-Pref SM" | risk_class == "Undiff NS" ) &
    duration == "06-09" & post_level_premium_structure == "ART" |
    risk_class == "Non-Pref NS" &
    duration == "10" & post_level_premium_structure == "ART",
    risk_class, "Super-Pref NS"),
  levels = levels_risk_class),
premium_mode_group = premium_mode %>% #cf) non #I-3
  factor(levels = levels_premium_mode,
    labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
               "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group = premium_mode %>% #I-3
  factor(levels = levels_premium_mode,
    labels = c("Annual", "Semiannual", "Quarterly",
               "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group_q = if_else( #I-3
  duration == "10" & premium_mode == "Quarterly", 1, 0),
premium_jump_ratio_group = cut(
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE),
premium_jump_ratio_group2 = cut( #I-3
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 5, 99), right = FALSE),
premium_jump_ratio_rc_NPNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration >= "11" & post_level_premium_structure == "Other") &
  risk_class == "Non-Pref NS", premium_jump_ratio, 0),
premium_jump_ratio_rc_PBNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration == "12" & post_level_premium_structure == "Other") &
  risk_class == "Pref Best NS" & (duration >= "10"), premium_jump_ratio, 0),
face_amount_d09PrUnHi = if_else( #I-9 (adjusted to 2006-train)
  duration == "06-09" & post_level_premium_structure == "ART" &
  premium_jump_ratio_unknown == 1 & face_amount == "1M-",
  1, 0)
) %>%
select(duration, post_level_premium_structure,

```

```

        issue_age, issue_age_sq, issue_age_log, risk_class_group,
        issue_age_70, #I-5
        issue_age_d10ART, #I-8
        risk_class_int, #I-6
        premium_mode_group, premium_jump_ratio_group,
        premium_mode_oo, #I-4
        premium_jump_ratio_group2, premium_mode_group_q, #I-3
        premium_jump_ratio_rc_NPNS, premium_jump_ratio_rc_PBNS, #I-7
        face_amount, #I-1
        face_amount_d09PrUnHi, #I-9
        gender, #I-10
        exposure, lapse)
}

```

[JA] モデル構築のステップでは、いくつかの交互作用項を明示的に追加するのみである。前処理で追加された説明変数は formula の . 指定で自動的にモデルに追加される。

[EN] English Here.

```

df_train_2006_mod <- df_train_2006 %>% prepfun_mod_xy
df_test_2006_mod <- df_test_2006 %>% prepfun_mod_xy

model_mod_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age
     -premium_jump_ratio_group2 -premium_mode_group_q #I-3
     + premium_jump_ratio_group2 : premium_mode_group_q #I-3
     + risk_class_int * issue_age #I-6
     + issue_age_d10ART : face_amount #I-8
     - 1) : duration : post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_mod)

```

## 3 Statistical Predictive Performance

### 3.1 Reduced Baseline GLM

[JA] Reduced Baseline GLM のスコアは次のとおり。関数 `calc_logloss()` の引数 `mean` を `FALSE` とした場合は Log-loss の総和が、`TRUE` とした場合は加重平均値が出力される。元論文の I-1 の節で表示した数値がここで再現される。

[EN] English Here.

```

pred_soa <- model_soa_2006 %>%
  predict(df_test_2006_soa %>% prepfun_soa_xy_test_2006, type = c("response"))

calc_logloss(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)

```

```
)#0.2988156
```

```
[1] 0.2988156
```

```
calc_logloss(  
  pred_soa,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = FALSE  
)#238153.1
```

```
[1] 238153.1
```

```
calc_auc(  
  pred_soa,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.8065347
```

```
[1] 0.8065347
```

## 3.2 XGBoost Model

[JA] XGBoost のスコアは次のとおり。元論文の Statistical Predictive Performance の節で示した数値 (Validation Year は 2007) が再現される。

[EN] English Here.

```
df_test_2006_mm <- df_test_2006 %>% get_mm  
df_test_2006_mm <- df_test_2006_mm[, -constant_cols_2006, drop = FALSE]  
pred_xgb <- model_xgboost_2006 %>% predict(df_test_2006_mm)  
  
calc_logloss(  
  pred_xgb,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = TRUE  
)#0.2935507
```

```
[1] 0.2935507
```

```
calc_logloss(  
  pred_xgb,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = FALSE  
)#233957.1
```

```
[1] 233957.1
```

```
calc_auc(  
  pred_xgb,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.820604
```

[1] 0.820604

### 3.3 Baseline GLM

[JA] XGBoost のスコアは次のとおり。Section 2.1 で注意したように、元論文の Statistical Predictive Performance の節で示した数値からは、Log-loss で僅かにずれが生じる。

[EN] English Here.

```
pred_modi1 <- model_modi1_2006 %>%  
  predict(df_test_2006_modi1 %>% prepfun_soa_xy_test_2006, type = c("response"))  
  
calc_logloss(  
  pred_modi1,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = TRUE  
)#0.2984396
```

[1] 0.2984396

```
calc_logloss(  
  pred_modi1,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = FALSE  
)#237853.5
```

[1] 237853.5

```
calc_auc(  
  pred_modi1,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.8073218
```

[1] 0.8073218

### 3.4 Improved GLM

[JA] XGBoost のスコアは次のとおり。Section 2.1 で注意したように、元論文の Statistical Predictive Performance の節で示した数値からは、Log-loss で僅かにずれが生じる。なお、Baseline GLM と比べて Log-loss ではわずかに悪化している点は元論文と同様である。

[EN] English Here.

```

pred_mod <- model_mod_2006 %>%
  predict(df_test_2006_mod %>% prepfun_soa_xy_test_2006, type = c("response"))

calc_logloss(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2984574

```

[1] 0.2984574

```

calc_logloss(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#237867.7

```

[1] 237867.7

```

calc_auc(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.8090849

```

[1] 0.8090849

## Acknowledgments

**[JA]** 本研究の実施にあたり、公表データ（SOA 10-Year Level Premium Term Lapse Study）を提供いただいた Society of Actuaries (SOA) に深謝いたします。なお、本論文の作成プロセスにおける英文表現の推敲およびコード構築の補助にあたっては、適切な人間による確認と責任のもとで生成 AI ツール（Gemini / ChatGPT 等）を活用いたしました。

**[EN]** The authors gratefully acknowledge the Society of Actuaries (SOA) for providing the public dataset (SOA 10-Year Level Premium Term Lapse Study) utilized in this study. In addition, generative AI tools (such as Gemini and ChatGPT) were used to assist with English language editing and R code refinement, with all final content thoroughly reviewed and verified by the authors.

## References

Society of Actuaries. (2014). *2014 post level term lapse & mortality report*. <https://www.soa.org/resources/experience-studies/2014/research-2014-post-level-shock/>